Lab 2 Report — Signal Processing

Group members: Yacoub Dawli (Jakob) — yada2100@student.miun.se
Obid Alrhman Aljabi — obal2302@student.miun.se 22 October 2026

1. Introduction

In this lab, we looked at signals in both the time and frequency domains. We tested how different windows and filters change a signal, and we used recordings of our own names to see how sound changes over time.

2. Experimental Setup

We used Python with NumPy, SciPy and Matplotlib. The main signal was made from three cosine waves at 50, 55 and 120 Hz, sampled at 250 Hz for 0.6 seconds (150 samples). We also tested two digital systems and designed filters at a sampling rate of 2,500 Hz. For the voice task, we analyzed separate WAV recordings from both group members.

3. Results and Analysis

3.1 Time-domain analysis

The time plot shows the three waves added together. The FFT shows peaks at 50, 55 and 120 Hz. The full spectrum also has matching peaks at negative frequencies. This is normal for a real cosine signal; the negative side does not mean extra sounds were added.

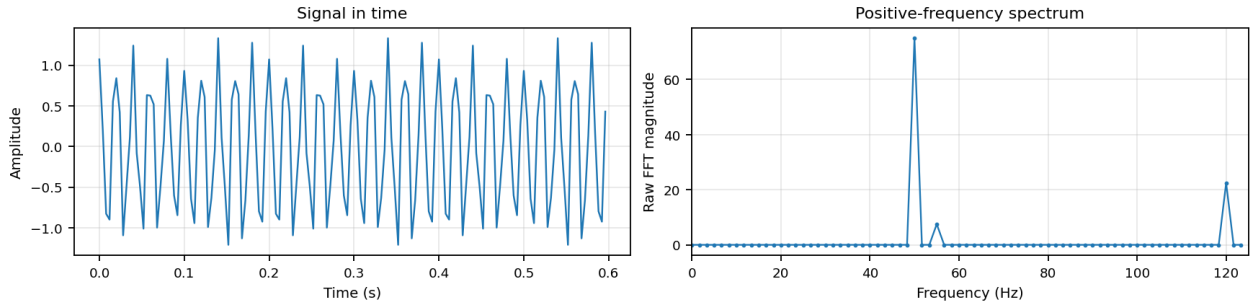


Figure 1. Original signal and its positive-frequency spectrum.

3.2 Frequency identification

We compared rectangular, Hann and Hamming windows. For this signal, the rectangular window separated 50 and 55 Hz most clearly. Its peaks were narrower. Hann and Hamming made the peaks wider but generally help reduce leakage. The rectangular result is especially clear here because the frequencies fall exactly on FFT frequency points.

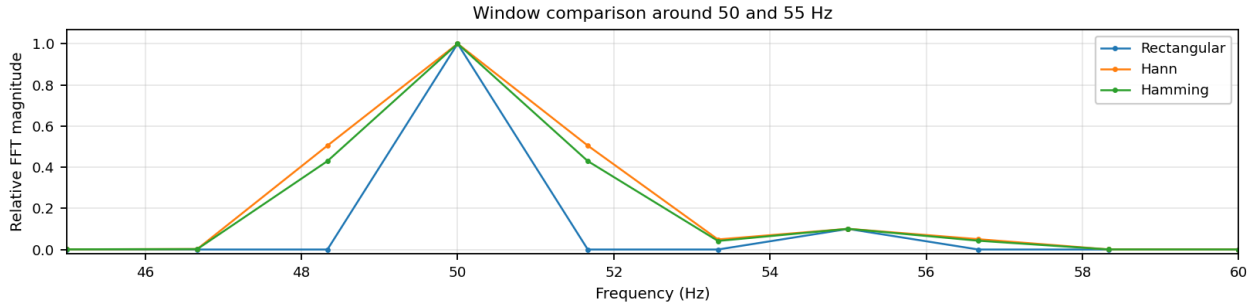


Figure 2. The three windows around the nearby 50 and 55 Hz frequencies.

3.3 Linear time-invariant systems

For System 1, convolution gave the eight values in the given impulse response, followed by zeros. This is an FIR system because its impulse response ends. For System 2, we used a loop and SciPy's lfilter with b = [1, 0.3] and a = [1, 0.6, −0.35]. Both methods gave the same 100 output samples (maximum difference: 0). System 2 is IIR because previous output values are used to calculate the next ones.

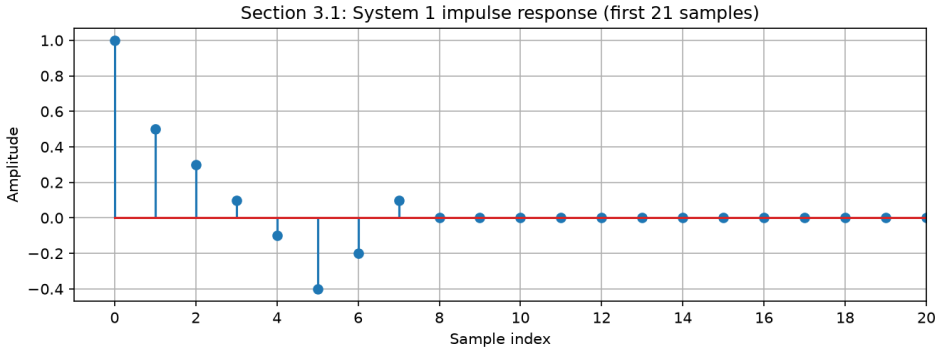


Figure 3. System 1 impulse response: eight nonzero samples, then zeros.

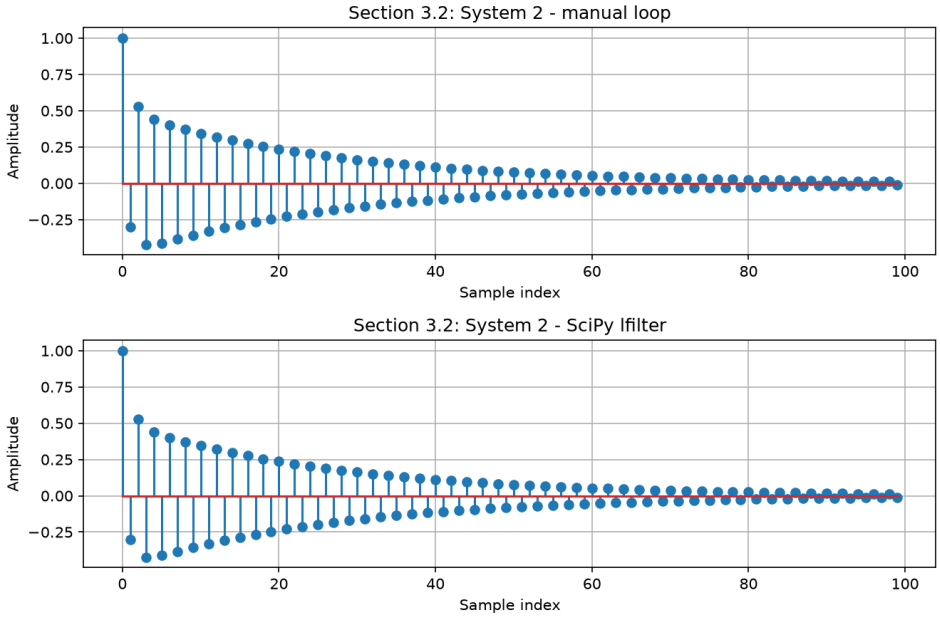


Figure 4. System 2: matching 100-sample responses from the loop and SciPy lfilter.

3.4 Frequency-domain analysis

We read the frequencies, amplitudes and phases from the FFT. After scaling the FFT magnitudes, all three amplitudes and phases agreed with the original signal. Only the phases at the three significant frequency components are shown, since phases at near-zero FFT magnitudes are not meaningful.

Frequency	Amplitude	Phase
50 Hz	1.0	0 rad
55 Hz	0.1	0.785 rad (π/4)
120 Hz	0.3	1.571 rad (π/2)

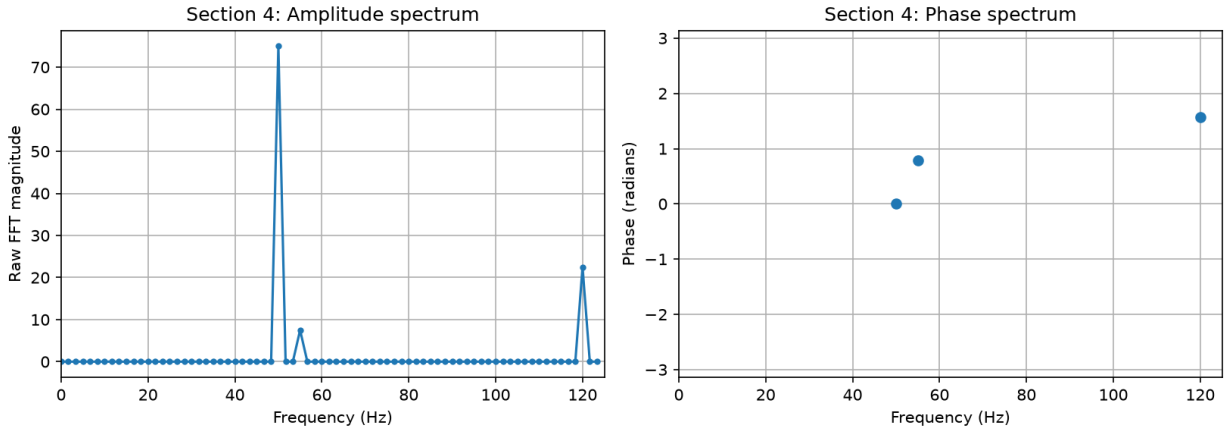


Figure 5. The signal's amplitude spectrum (raw FFT magnitude) and phase spectrum.

3.5 System frequency response

We passed a 50 Hz cosine through System 2. Its amplitude dropped from 1.0 to 0.7481 and its phase shifted to −0.0103 radians. The system response predicted an amplitude of 0.7454 and a phase of −0.0128 radians. The amplitude difference was about 0.37%. This is small and is explained by the short recording and the filter's start-up behavior. The result supports Y(f) = H(f)X(f).

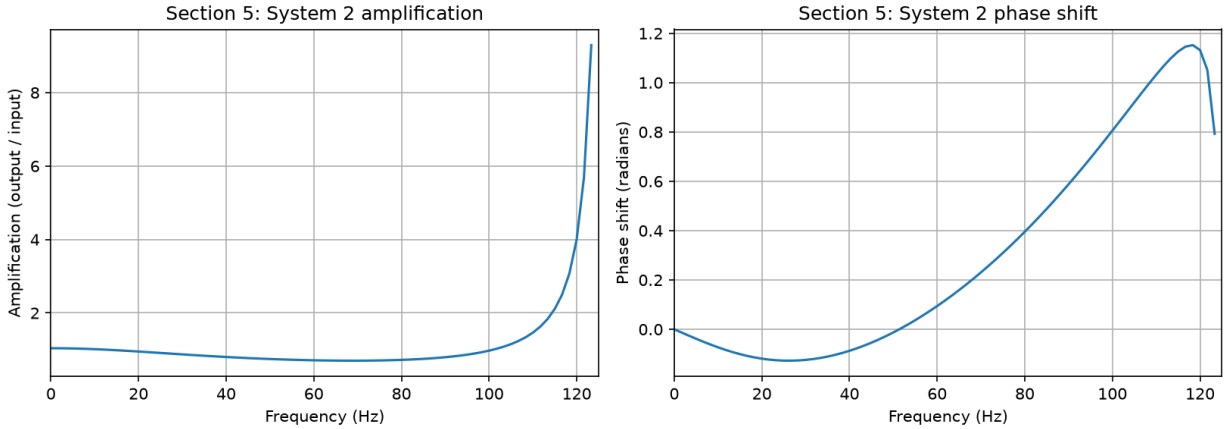


Figure 6. System 2 amplification and phase shift as functions of frequency.

3.6 Filtering

Butterworth had a smooth response. Chebyshev Type I had about 3 dB of ripple in the passband but dropped faster after the cutoff. At 300 Hz, their levels were −18.9 dB and −37.4 dB. A low-pass filter keeps lower frequencies, while a high-pass filter keeps higher ones. Raising the cutoff from 200 to 350 Hz passed more high frequencies. Raising the order from 3 to 7 made the cutoff sharper. The phase changed too. At 100 Hz, raising the cutoff from 200 to 350 Hz changed the phase from −1.649 to −0.877 radians. Increasing the order from 3 to 7 changed it from −1.033 to −2.282 radians.

We also made band-pass and band-stop filters for 150–400 Hz. The band-pass kept this range, while the band-stop reduced it.

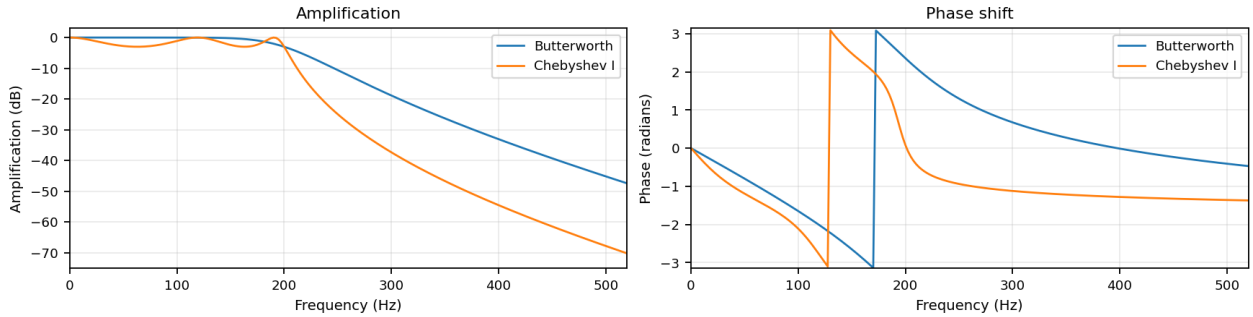


Figure 7. Butterworth and Chebyshev Type I low-pass filters (order 5, cutoff 200 Hz).

3.7 Frequency-time analysis of our voices

We recorded our names separately. Jakob's recording was 1.685 seconds and Obid's was 2.240 seconds; both used 48,000 Hz. We split each recording into short parts and calculated the FFT of each part. This is the short-time Fourier transform, X(t,f). Its squared magnitude gives the power shown in a spectrogram.

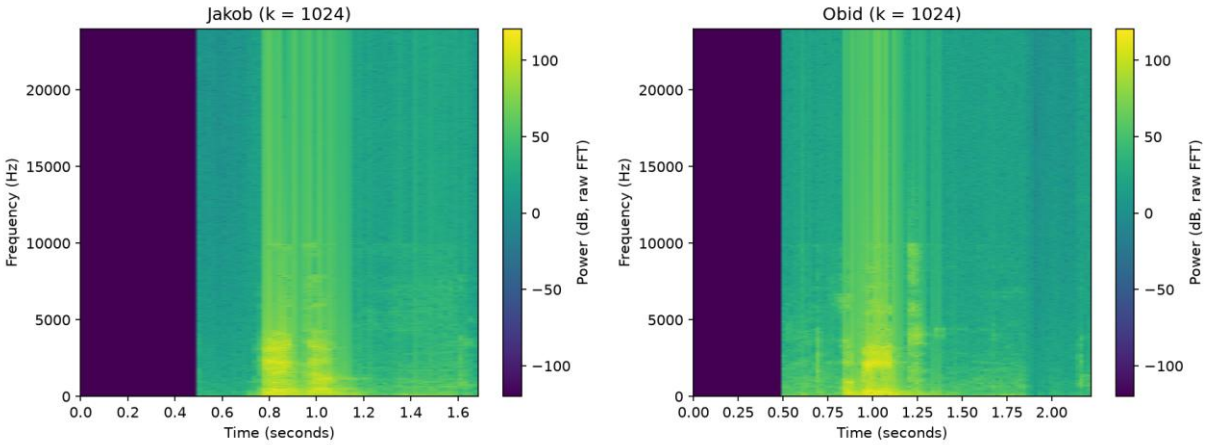


Figure 8. Our separate voice spectrograms, using 1,024 samples per part.

Jakob's strongest areas appear around 0.8–1.1 seconds, mainly below 4,000 Hz. Obid's strongest areas are around 0.85–1.25 seconds, with some energy reaching higher frequencies. We could see different sound regions, but could not reliably assign exact letters or syllables from these plots alone.

Samples per part	Time per part	Frequency spacing
512	10.67 ms	93.75 Hz
1,024	21.33 ms	46.88 Hz
2,048	42.67 ms	23.44 Hz

Smaller parts made quick changes easier to place in time. Larger parts showed frequencies more precisely, but gave less detail about when sounds changed.

4. Discussion

The results show the trade-offs clearly. A window that makes peaks easier to separate in one case will not always be the best choice. Filters can reduce or keep certain frequencies, but their settings also change phase. In the voice recordings, we could see when the sounds changed, although the plots alone were not enough to identify every sound.

5. Conclusion

We used FFTs to find signal frequencies and phases, compared FIR and IIR systems, and tested several filters. We also used spectrograms to compare our own recordings. The measurements matched the expected behavior, with only small differences in the finite-length filter test.

Appendix: Code Snippets

These are the main Python commands used in the lab:

```
X = fft(x)                          # Frequency spectrum
phase_x = np.angle(X)              # Phase spectrum
y3 = lfilter(b, a, x2)           # Filter output
b_b, a_b = signal.butter(5, 200, btype="low", fs=2500)
band = [150, 400]
b_bp, a_bp = signal.butter(5, band, btype="bandpass", fs=2500)
b_bs, a_bs = signal.butter(5, band, btype="bandstop", fs=2500)
Xvoice = np.fft.fft(xm_xvoice, axis=1)
power_spectrum = np.abs(Xvoice) ** 2  # Voice power
```

The FFT finds the frequencies in the signal, while np.angle() gives their phases.

lfilter() calculates the system output. The Butterworth commands set up a low-pass filter and filters that keep or remove the 150–400 Hz band.

For the voice recordings, axis=1 applies the FFT to each short part. Squaring its magnitude gives the power shown in the spectrogram.

Source: DT081A Lab 2 instructions and course lectures 4–7. Full code and other figures are kept with the lab files.